LECTURE 5

Arrays with Int/Double/char Datatypes, Multi-dimensional arrays; Sorting and Searching problems

RECAP

```
do{
    // This block is repeatedly executed as long as expression is TRUE
    block-of-code
}while(logical-expression);
```

RECAP

```
while(logical-expression){
    // This block is repeatedly executed as long as expression is TRUE
    block-of-code
}
```

RECAP

```
for(init_expr;logical_expr;increment_expr){
    // This block is repeatedly executed as long as expression is TRUE
    block-of-code
}
```

ARRAYS

int double char ARRAYS

MOTIVATION

- STORE **FIXED**, YET UNKNOWN, OR LARGE NUMBER OF VALUES
  - E.G., STUDENT MARKS
-

MOTIVATION

- STORE **FIXED**, YET UNKNOWN, OR LARGE NUMBER OF VALUES
  - E.G., STUDENT MARKS, RECORD/COLUMN IN A TABLE, ETC.
-

student1	**90**
student2	95
student3	93
student4	100
student5	98

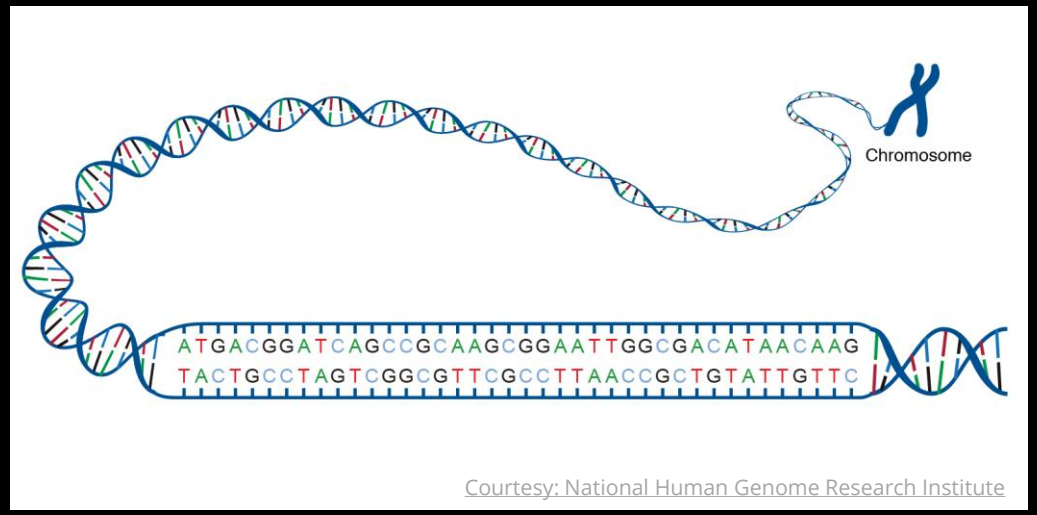
MOTIVATION

- STORE **FIXED**, YET UNKNOWN, OR LARGE NUMBER OF VALUES
  - E.G., STUDENT MARKS, RECORD/COLUMN IN A TABLE, ETC.
- EUCLIDEAN VECTORS, SYMBOLIC SEQUENCES

$$\left[\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right]$$

MOTIVATION

- STORE **FIXED**, YET UNKNOWN, OR LARGE NUMBER OF VALUES
  - E.G., STUDENT MARKS, RECORD/COLUMN IN A TABLE, ETC.
- EUCLIDEAN VECTORS, SYMBOLIC SEQUENCES



Courtesy: National Human Genome Research Institute

DEFINING AN ARRAY

- `double marks[547];    //Defines 547 doubles;variable names begin with marks`
-
-
-

DEFINING AN ARRAY

- `double marks[547];`
- `int frequency[95];`
- `char word[45];`

- *`datatype`* `name[`*`length_exp`*`];`

ACCESSING ELEMENTS OF AN ARRAY

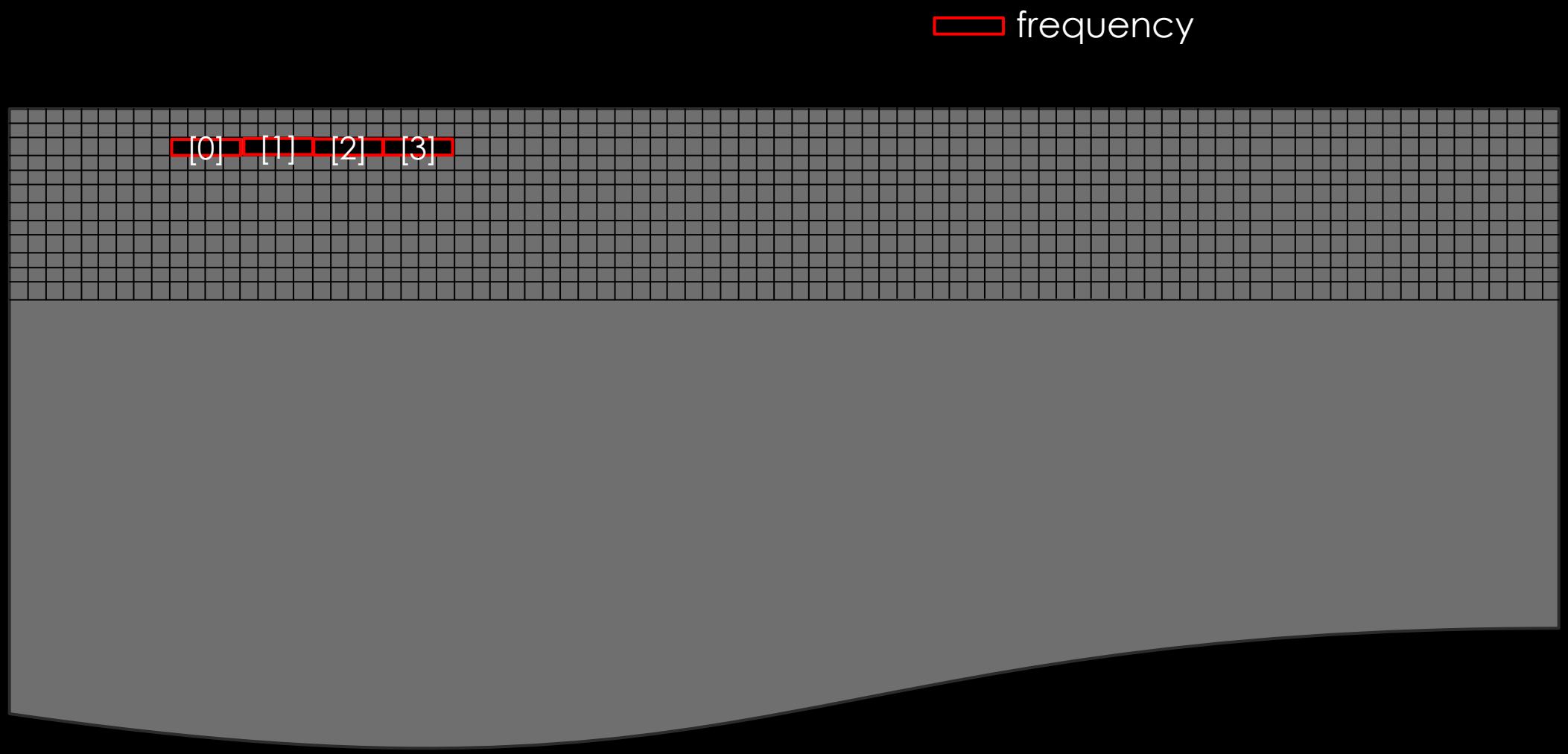
- `double marks[547];//marks[0] is the first element in marks array`
- `int frequency[95];//frequency[10] is the` 11^{th} `element in frequency array`
- `char word[45];    // word[44] is the last element in word array`

- *`datatype`* `name[`*`Length_exp`*`]; //name[i] is the` $(i+1)^{th}$ `element in the array`
  `//Each element is of the same` *`datatype`*

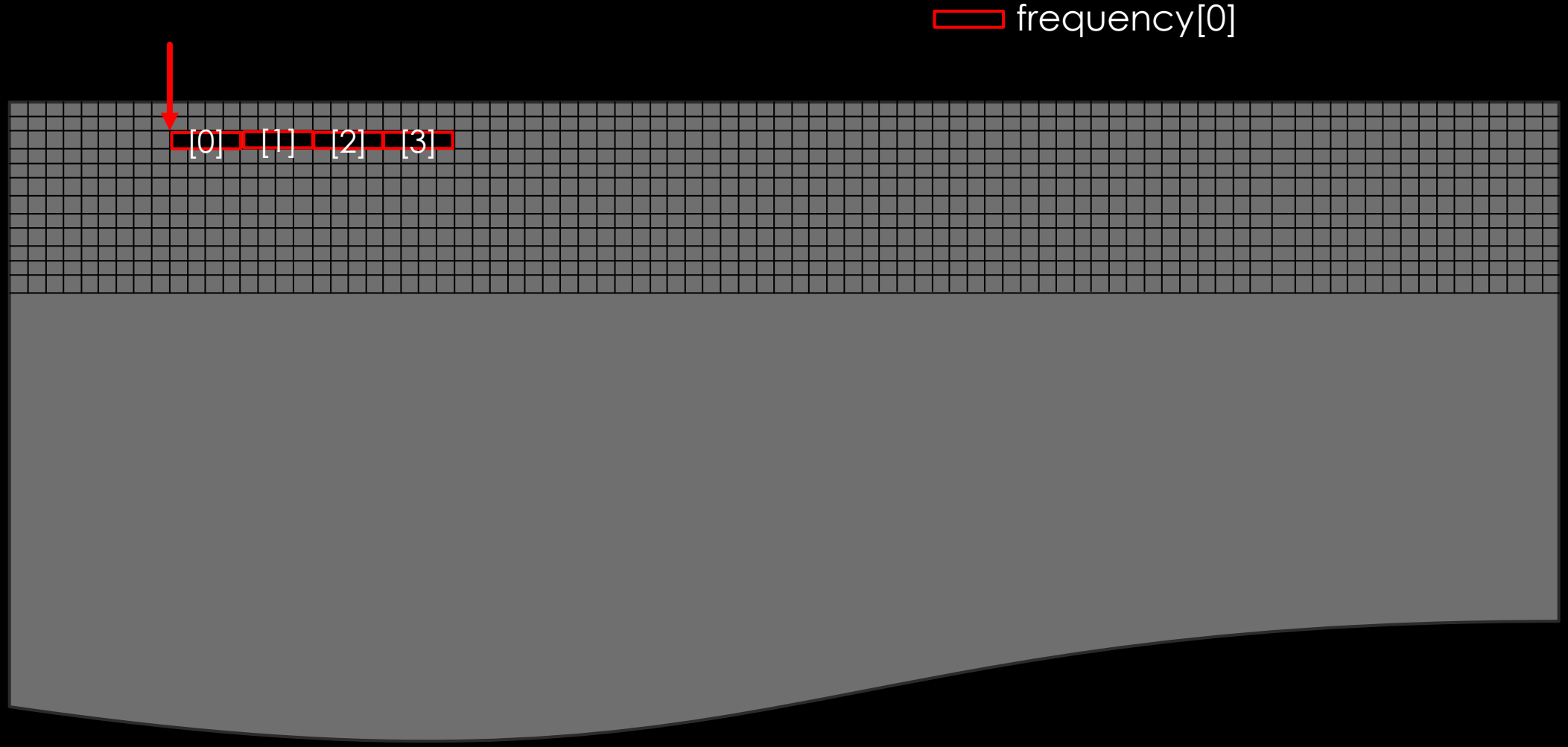
PRINTING/SCANNING ELEMENTS OF AN ARRAY

- `double marks[547];`
  - `printf("%lf",marks[40]);//prints the` 41^{st} `element in marks array`
  - `scanf("%lf",&marks[28]);//store keyboard input as` 29^{th} `element in marks array`

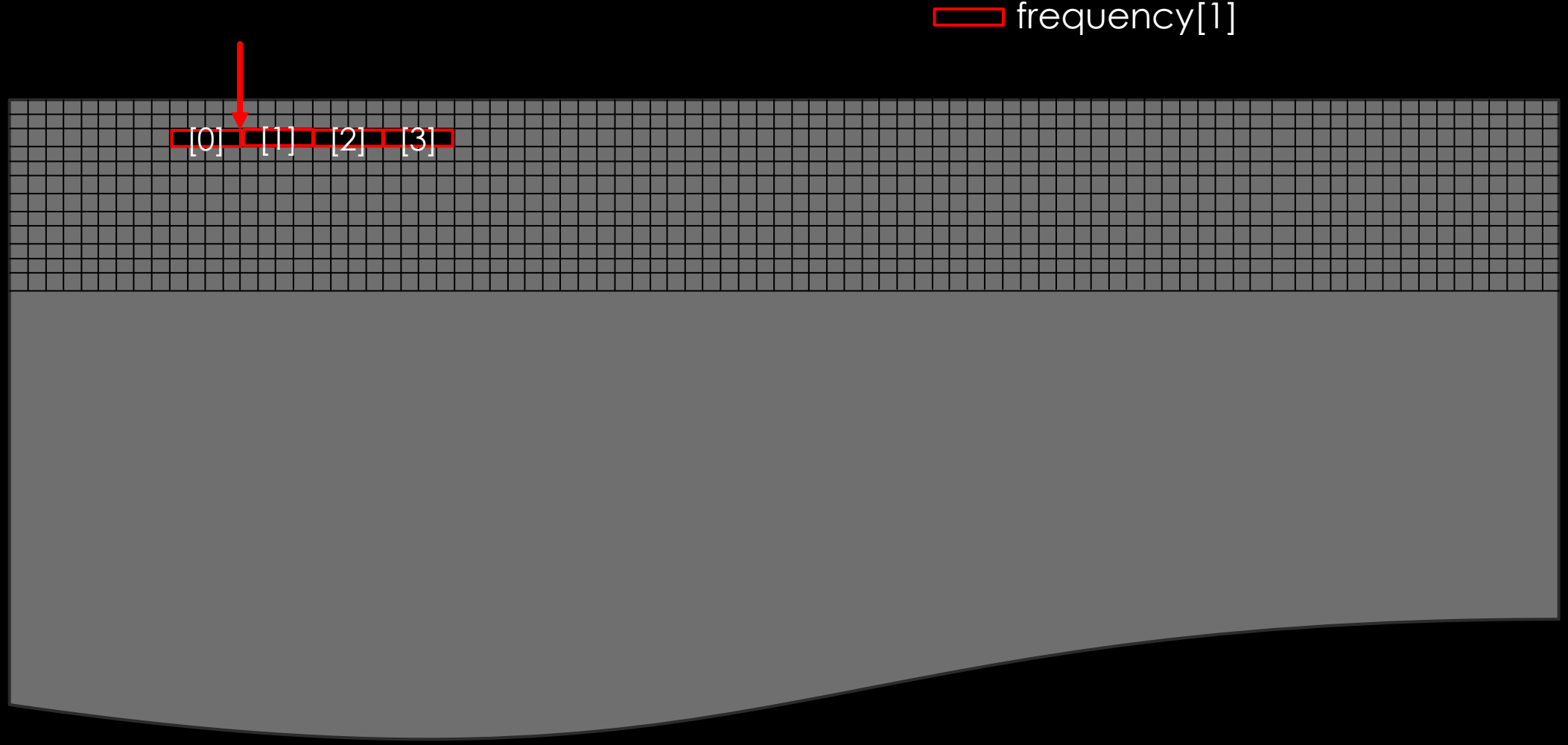
BEHIND THE SCENES



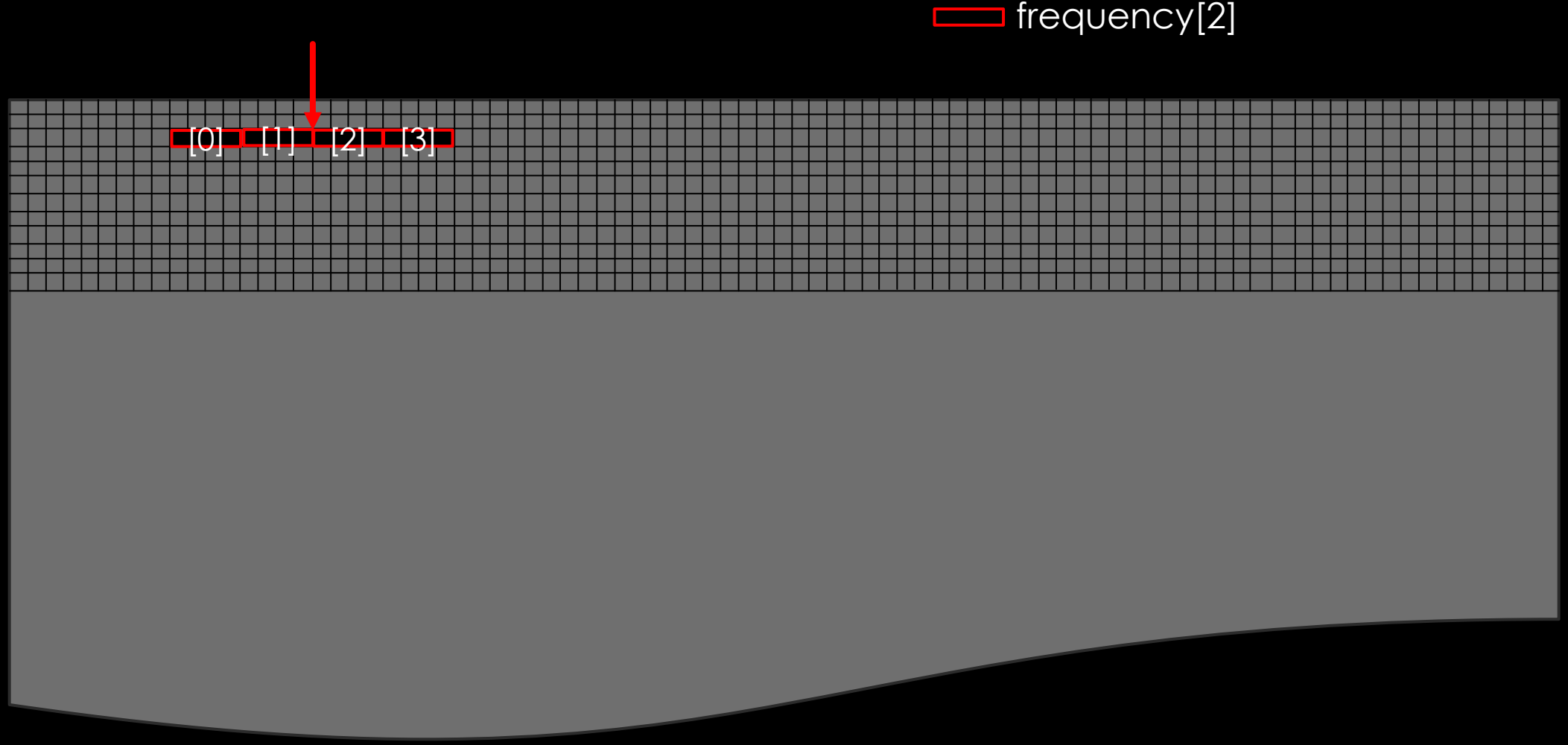
BEHIND THE SCENES



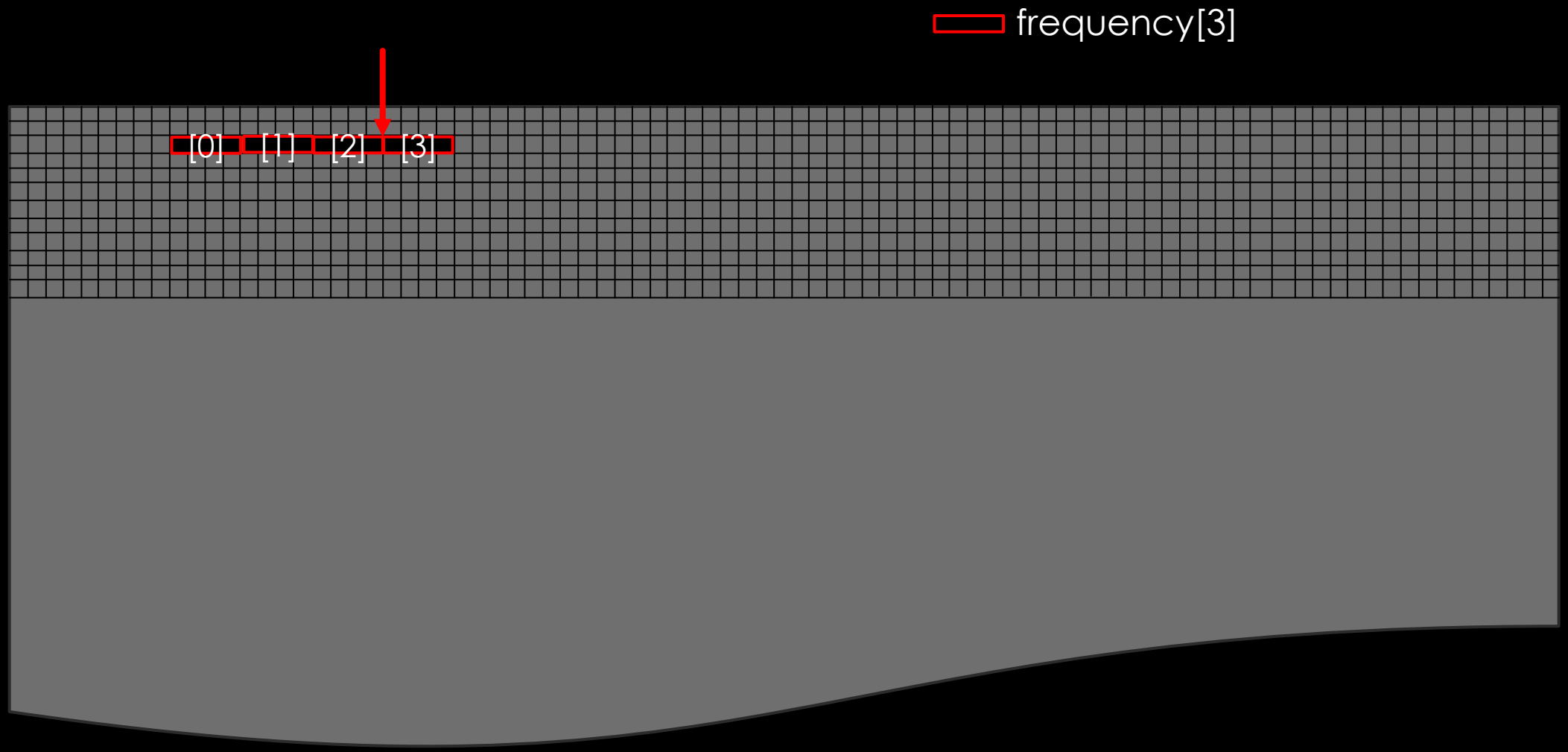
BEHIND THE SCENES



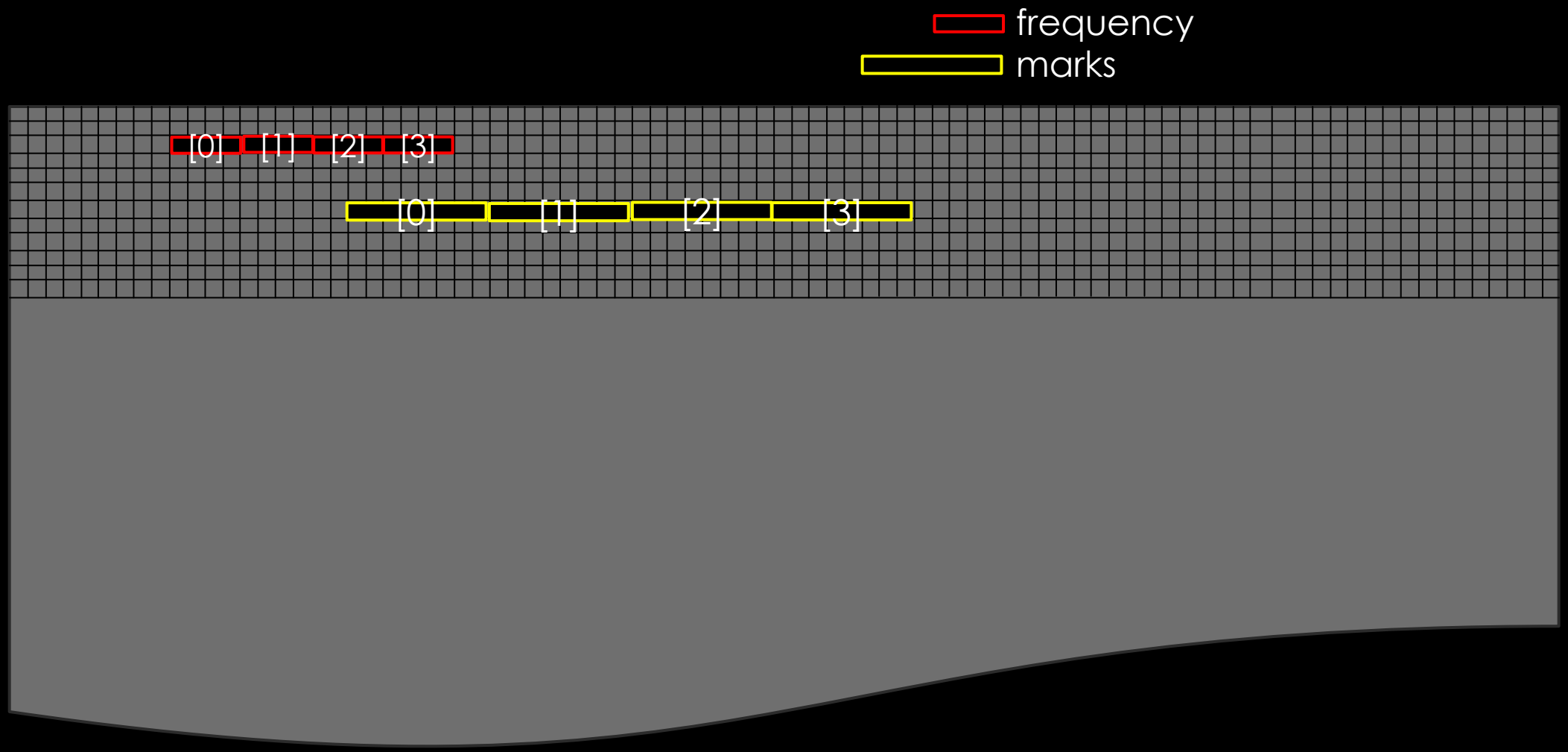
BEHIND THE SCENES



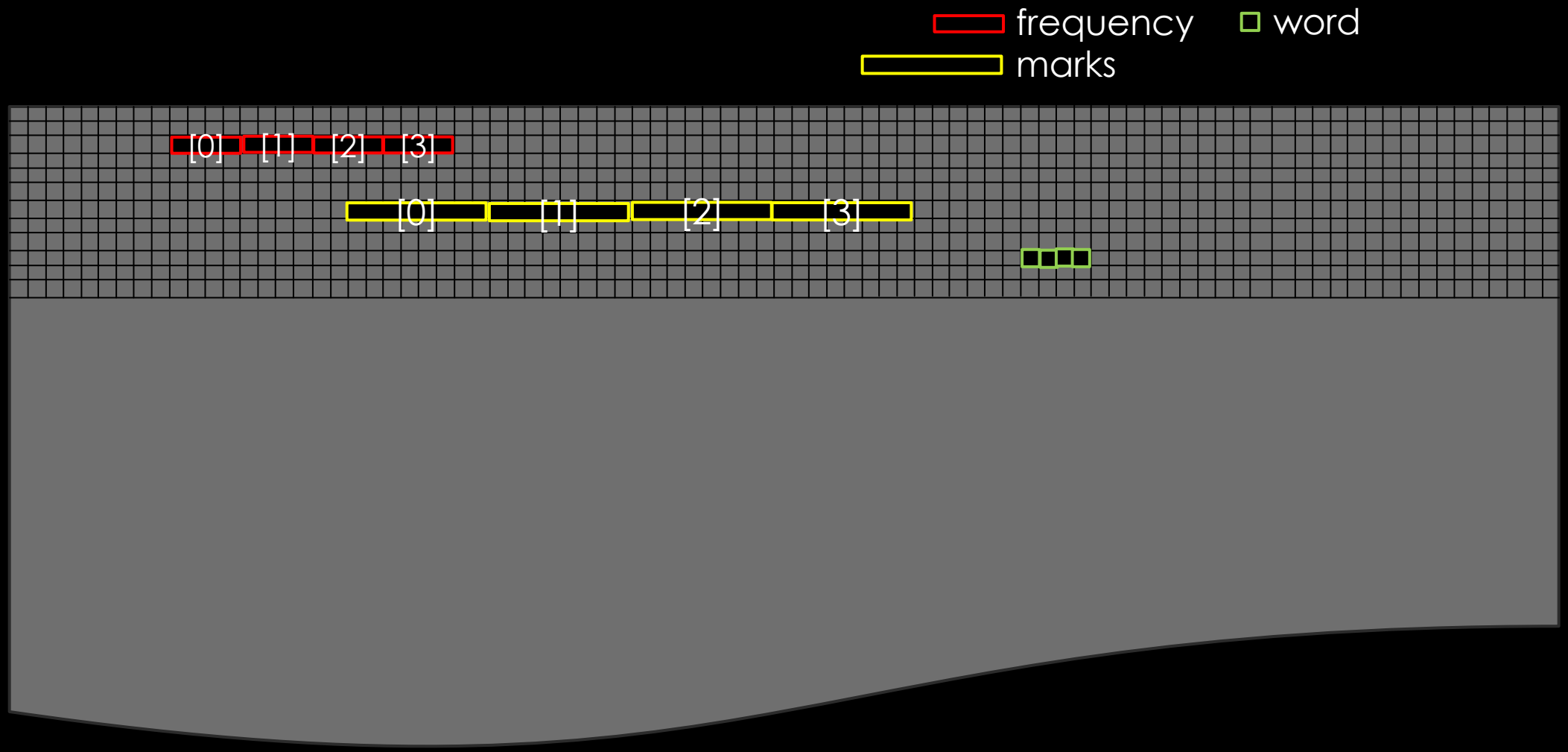
BEHIND THE SCENES



BEHIND THE SCENES



BEHIND THE SCENES



WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[0]);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[0]);     // prints garbage as not initialized
    return 0;
}
```

WHAT'S THE OUTPUT?

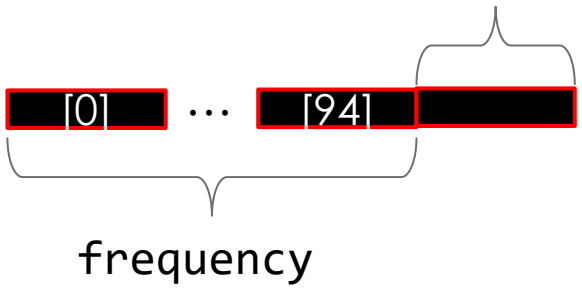
```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[95]);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[95]);// No compilation error. Assesses 96th address
    return 0;
}
```



WHAT'S THE OUTPUT?

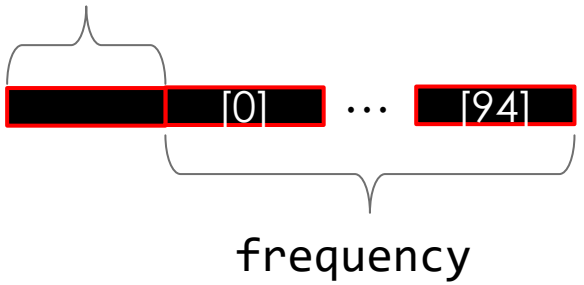
```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[-1]);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[-1]);//Assesses address before start. No compilation error.
    return 0;
}
```



WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[-400000]);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[95];
    printf("%d",frequency[-400000]);//Mostly runtime error (outside program block)
    return 0;
}
```

WHAT'S IN A NAME?

- NAME OF THE ARRAY ITSELF IS THE ADDRESS TO THE STARTING ELEMENT
  - `marks` IS SAME AS THE ADDRESS `&marks[0]`
- YOU CAN "MANUALLY" COMPUTE ADDRESSES AND INDEX
  - IS SAME AS (
    - +5 ADVANCES BY 8*5=40 BYTES
  - IS SAME AS (
    - +5 ADVANCES BY 4*5=40 BYTES
- YOU CAN USE THE SPECIFIER TO PRINT ADDRESSES

WHAT'S IN A NAME?

- NAME OF THE ARRAY ITSELF IS THE ADDRESS TO THE STARTING ELEMENT
  - `marks` IS SAME AS THE ADDRESS `&marks[0]`
- YOU CAN "MANUALLY" COMPUTE ADDRESSES AND INDEX
  - `marks[40]` IS SAME AS `(marks+5)[35]`
    - +5 ADVANCES BY 8*5=40 BYTES
  - IS SAME AS (
    - +5 ADVANCES BY 4*5=40 BYTES
- YOU CAN USE THE SPECIFIER TO PRINT ADDRESSES

WHAT'S IN A NAME?

- NAME OF THE ARRAY ITSELF IS THE ADDRESS TO THE STARTING ELEMENT
  - `marks` IS SAME AS THE ADDRESS `&marks[0]`
- YOU CAN "MANUALLY" COMPUTE ADDRESSES AND INDEX
  - `marks[40]` IS SAME AS `(marks+5)[35]`
    - +5 ADVANCES BY 8*5=40 BYTES
  - `frequency[40]` IS SAME AS `(frequency+5)[35]`
    - +5 ADVANCES BY 4*5=20 BYTES
- YOU CAN USE THE SPECIFIER TO PRINT ADDRESSES

WHAT'S IN A NAME?

- NAME OF THE ARRAY ITSELF IS THE ADDRESS TO THE STARTING ELEMENT
  - `marks` IS SAME AS THE ADDRESS `&marks[0]`
- YOU CAN "MANUALLY" COMPUTE ADDRESSES AND INDEX
  - `marks[40]` IS SAME AS `(marks+5)[35]`
    - +5 ADVANCES BY 8*5=40 BYTES
  - `frequency[40]` IS SAME AS `(frequency+5)[35]`
    - +5 ADVANCES BY 4*5=20 BYTES
- YOU CAN USE THE SPECIFIER `%p` TO PRINT ADDRESSES

DATA ENTRY INTO marks

```c
#include <stdio.h>

int main(void){
    double marks[547];
    for(unsigned short int i=0;i<=546;i++){
        printf("Enter marks of roll number %d: ",i+1);
        scanf("%lf",&marks[i]);
    }
    printf("Data entry done.\n");
    return 0;
}
```

DATA ENTRY INTO marks

```c
#include <stdio.h>

int main(void){
    double marks[547];
    for(unsigned short int i=0;i<=546;i++){
        printf("Enter marks of roll number %d: ",i+1);
        scanf("%lf",&marks[i]);      // scanf("%lf",marks+i);
    }
    printf("Data entry done.\n");
    return 0;
}
```

INITIALIZING ARRAYS USING LISTS

- `int frequency[5]={5,4,3,2,1};  //frequency[i]=5-i;`
-
-
-

INITIALIZING ARRAYS USING LISTS

- `int frequency[5]={5,4,3,2,1};  //frequency[i]=5-i;`
- `int frequency[5]={5};  //frequency[0]=5;rest to 0`
-
-

INITIALIZING ARRAYS USING LISTS

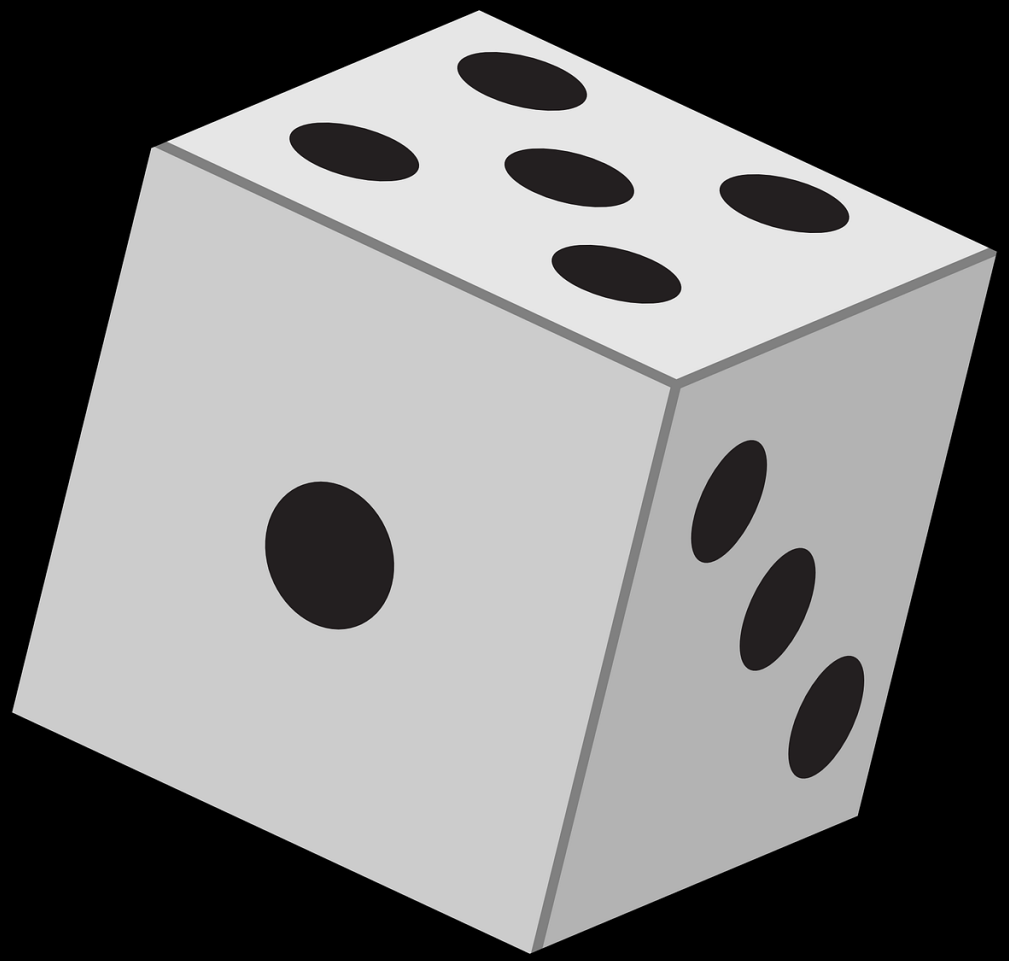
- `int frequency[5]={5,4,3,2,1};  //frequency[i]=5-i;`
- `int frequency[5]={5};  //frequency[0]=5;rest to 0`
- `int frequency[5]={5,4,3,2,1,0};// Compilation error`
-

INITIALIZING ARRAYS USING LISTS

- `int frequency[5]={5,4,3,2,1};  //frequency[i]=5-i;`
- `int frequency[5]={5};  //frequency[0]=5;rest to 0`
- `int frequency[5]={5,4,3,2,1,0};// Compilation error`
- `int frequency[]={5,4,3,2,1};  //same as first`

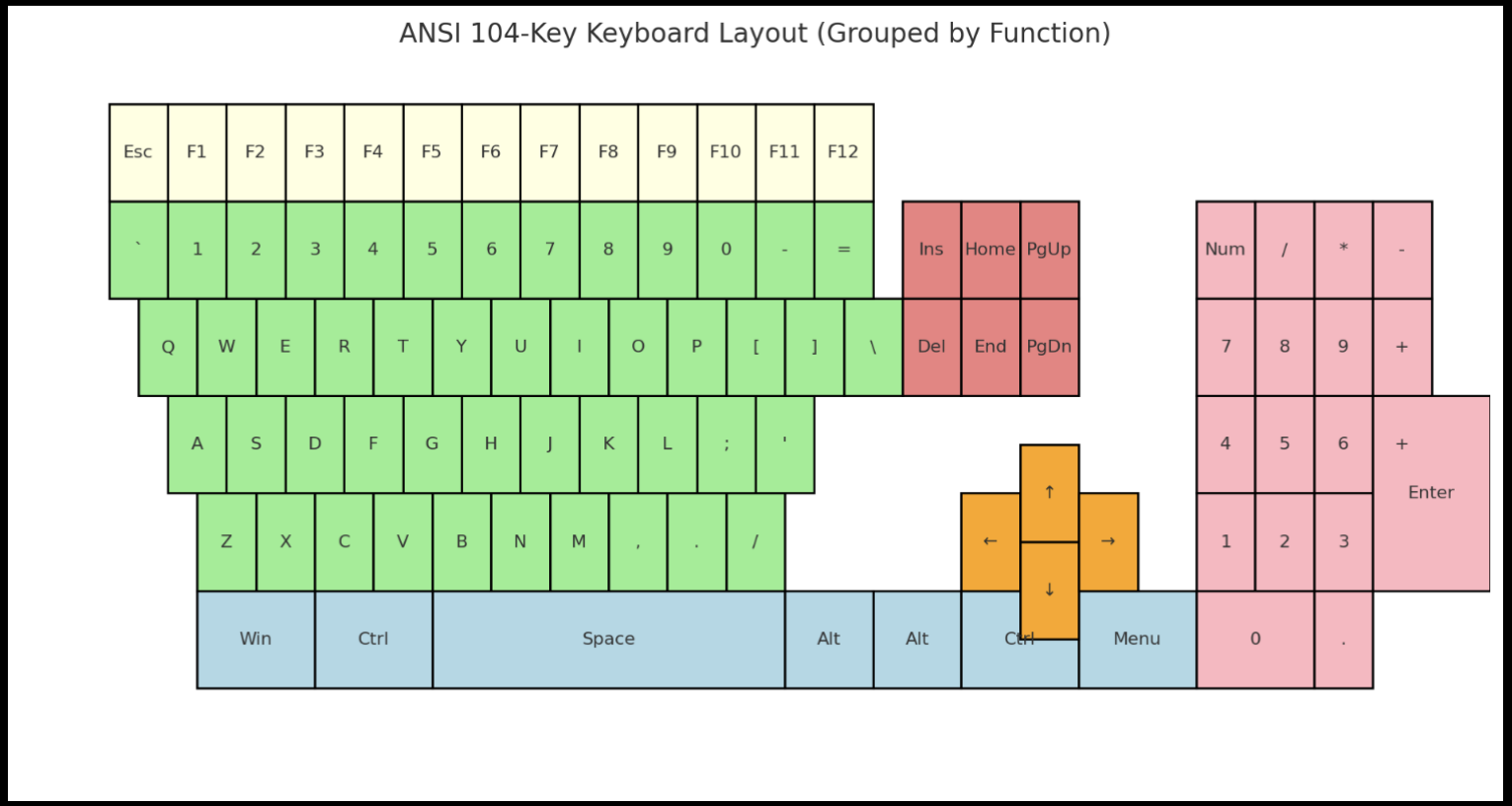
ESTIMATING PROBABILITIES

Roll few times



ESTIMATING PROBABILITIES

Random keyboard entry



ESTIMATING PROBABILITIES

```c
#include <stdio.h>
int main(void){
    unsigned int frequency[104]={0}, key, no_entries =1000;









}
```

ESTIMATING PROBABILITIES

```
#include <stdio.h>
int main(void){
    unsigned int frequency[104]={0}, key, no_entries=1000;
    for(unsigned int i=1;i<=no_entries;i++){
        printf("Enter face of %d roll: ",i); scanf("%d",&key]);
                                }




}
```

ESTIMATING PROBABILITIES

```c
#include <stdio.h>
int main(void){
    unsigned int frequency[104]={0}, key, no_entries=1000;
    for(unsigned int i=1;i<=no_entries;i++){
        printf("Enter face of %d roll: ",i); scanf("%d",&key]);
                            }  //Logic for updating frequency
    printf("FaceId\tProbability (estimated)\n");
    for(unsigned int i=0;i<=103;i++)
        printf("%d\t%.2lf\n",i+1,(double)frequency[i]/no_entries);
    return 0;
}
```

ESTIMATING PROBABILITIES

```c
#include <stdio.h>
int main(void){
    unsigned int frequency[104]={0}, key, no_entries=1000;
    for(unsigned int i=1;i<=no_entries;i++){
        printf("Enter face of %d roll: ",i); scanf("%d",&key]);
        frequency[key-1]++;}  // Update corresponding count
    printf("FaceId\tProbability (estimated)\n");
    for(unsigned int i=0;i<=103;i++)
        printf("%d\t%.2lf\n",i+1,(double)frequency[i]/no_entries);
    return 0;
}
```

WHAT'S THE OUTPUT?

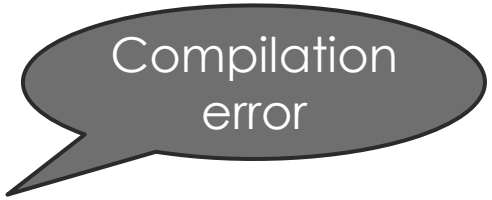
```c
#include <stdio.h>

int main(void) {
    int frequency[-95];
    printf("%p",frequency);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int frequency[-95];
    printf("%p",frequency);
    return 0;
}
```



WHAT'S THE OUTPUT?

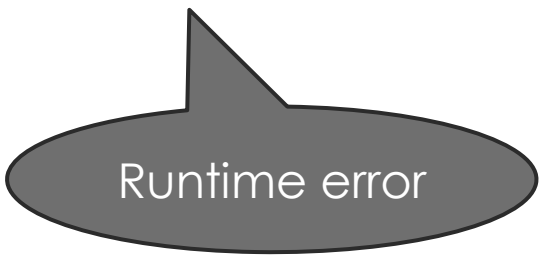
```c
#include <stdio.h>

int main(void) {
    int size; scanf("%d",&size); //input -9500
    int frequency[size];
    printf("%p",frequency);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>

int main(void) {
    int size; scanf("%d",&size); //input -9500
    int frequency[size];
    printf("%p",frequency);
    return 0;
}
```



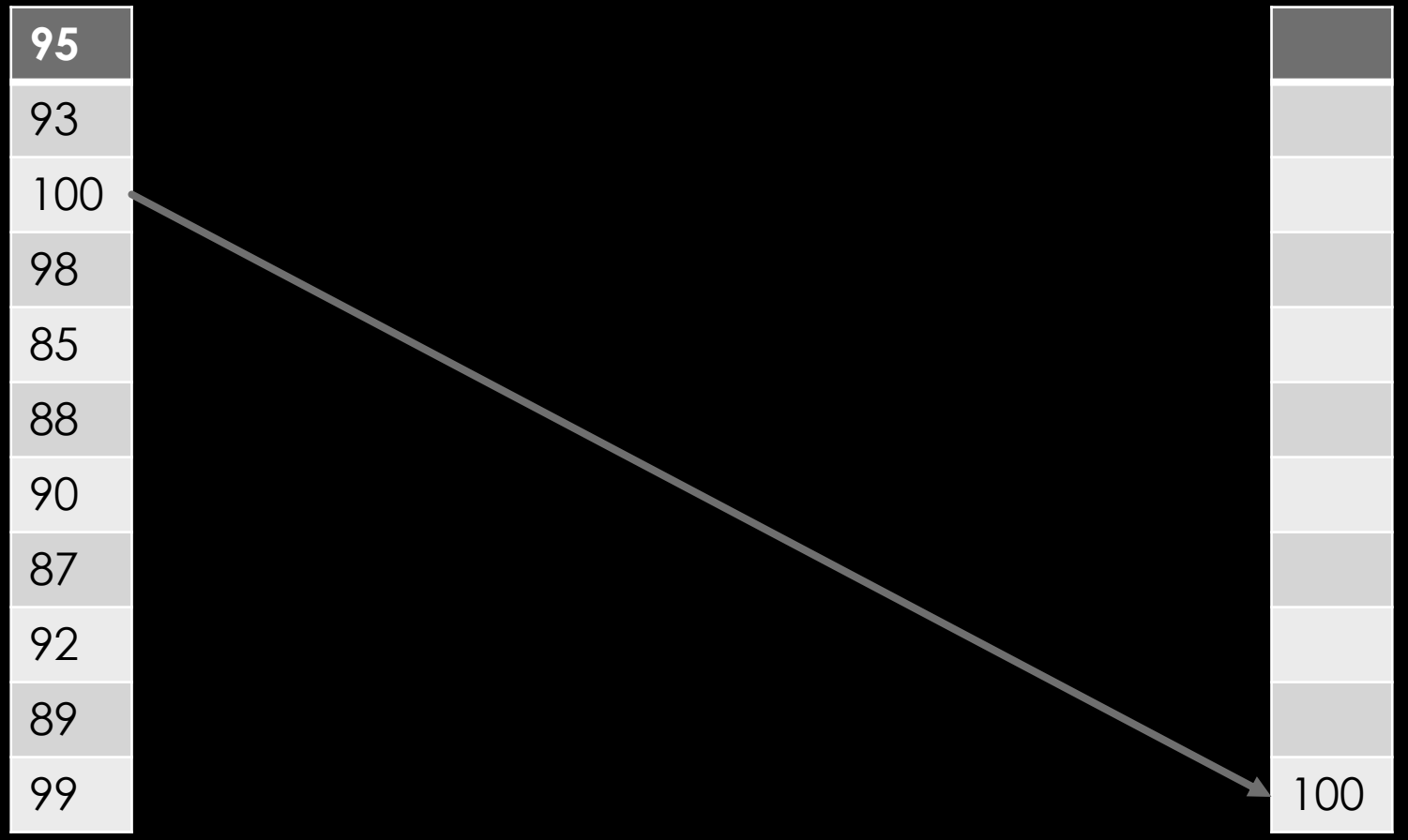
SORTING AND SEARCHING

USING LOOPS AND ARRAYS

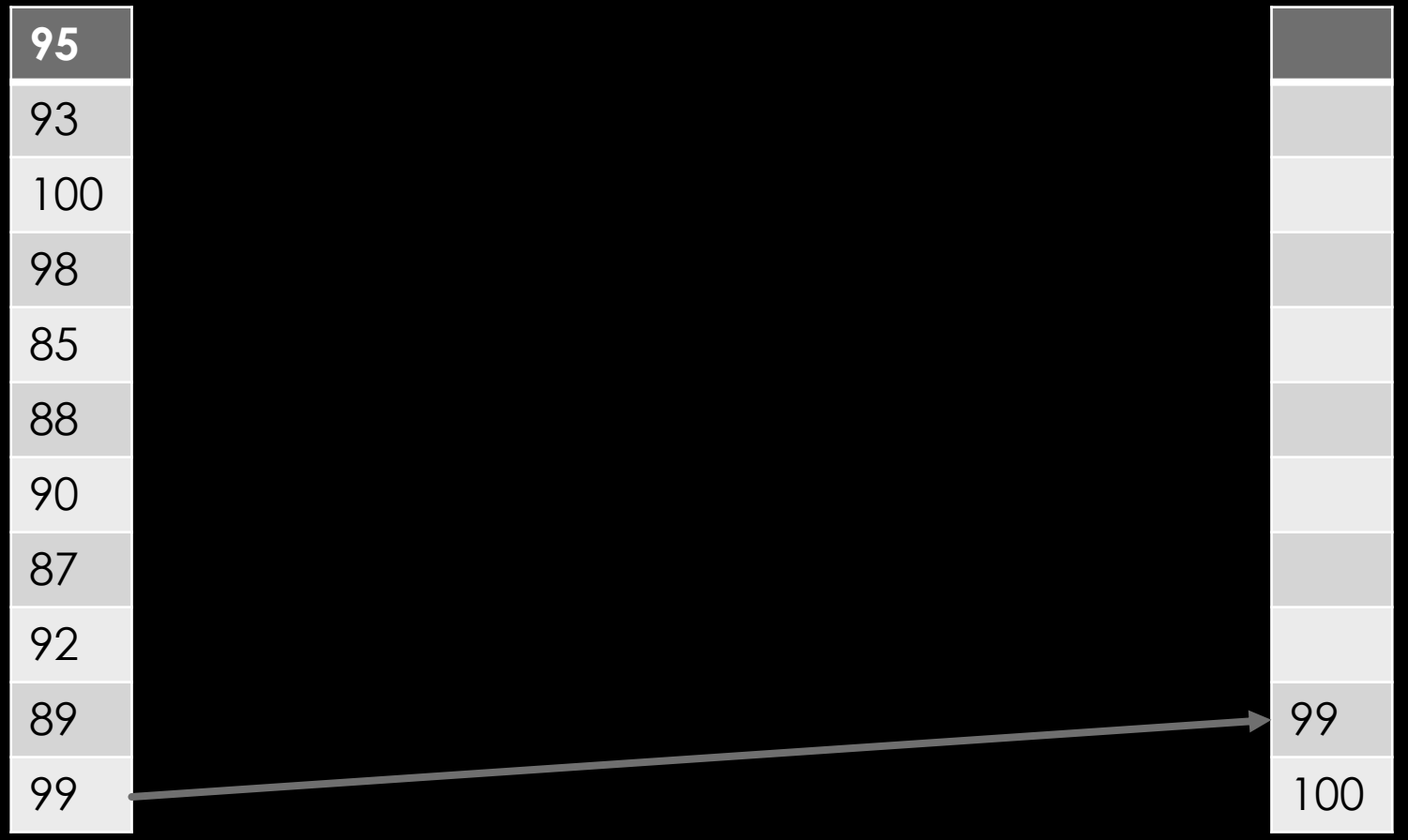
SORTING MARKS

95
93
100
98
85
88
90
87
92
89
99

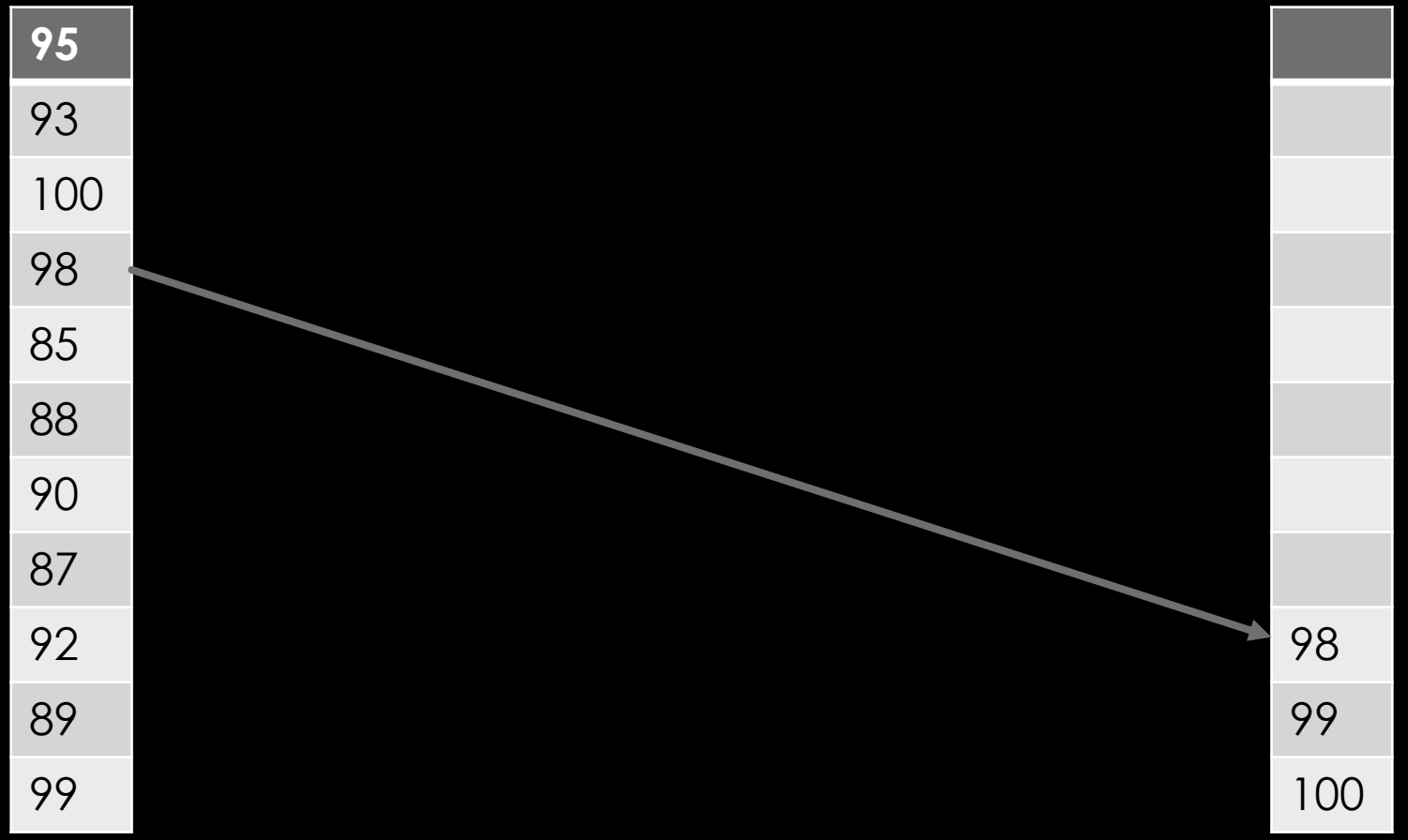
SORTING MARKS



SORTING MARKS



SORTING MARKS



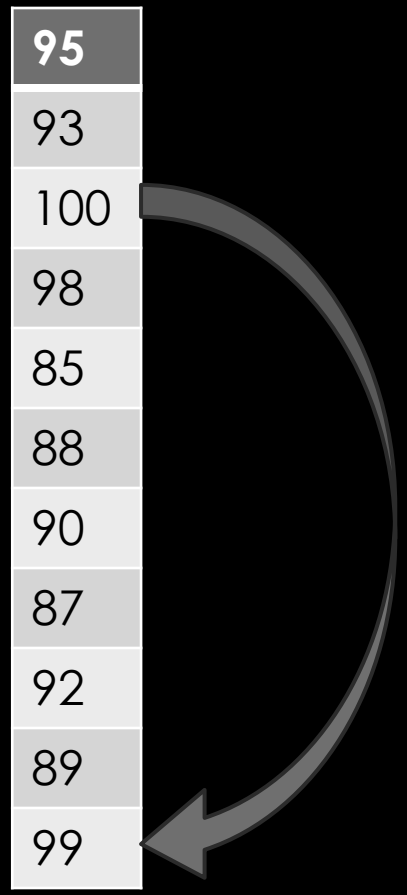
SORTING MARKS

1. Data entry of marks
2. Find max and store as last element
3. In remaining again find max and store as second last
4. Repeat until all marks are in order

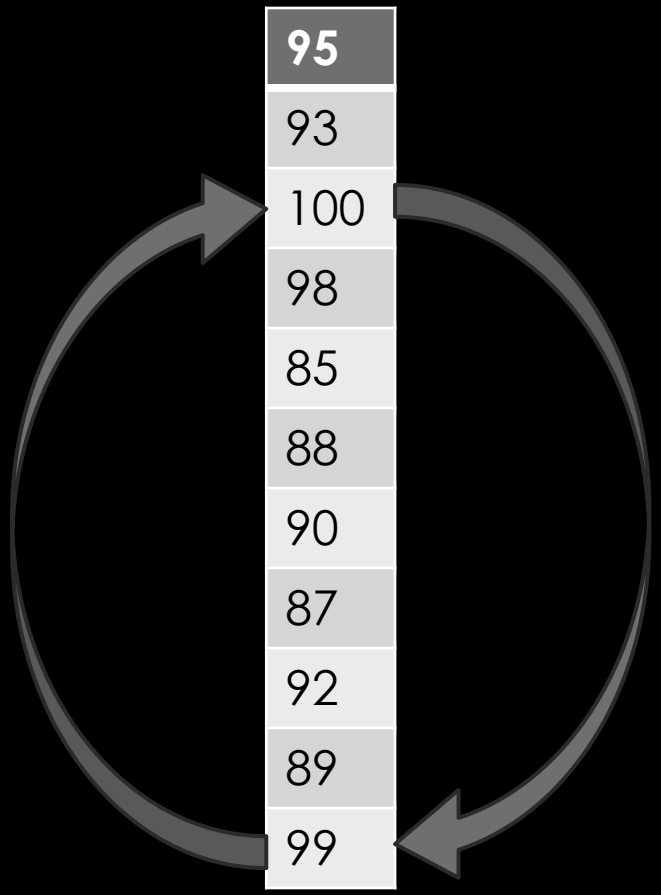
SORTING MARKS

95
93
100
98
85
88
90
87
92
89
99

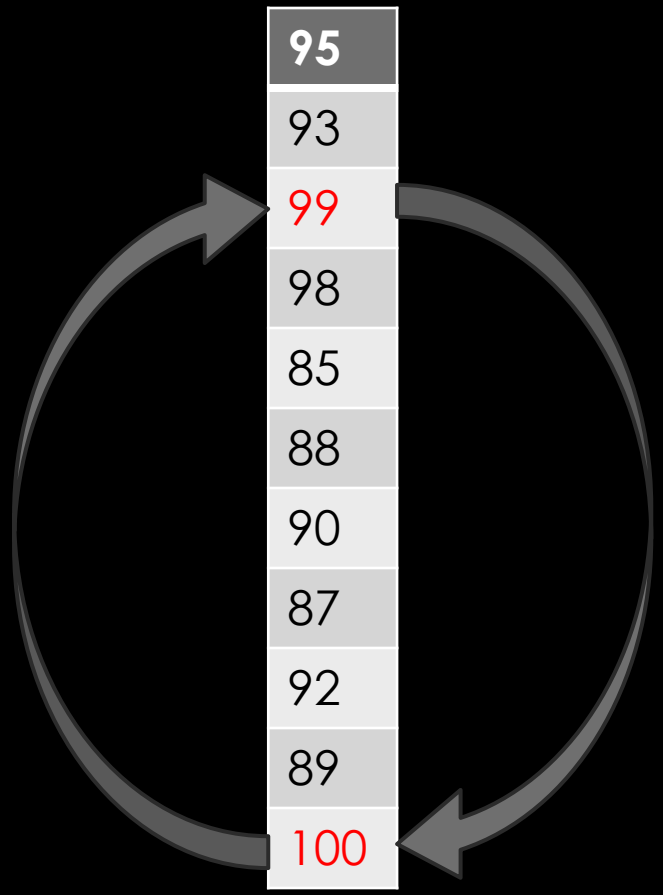
SORTING MARKS



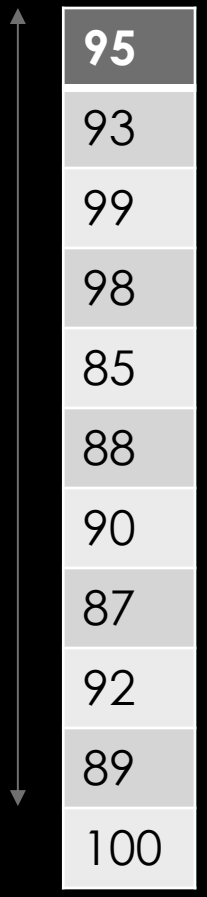
SORTING MARKS



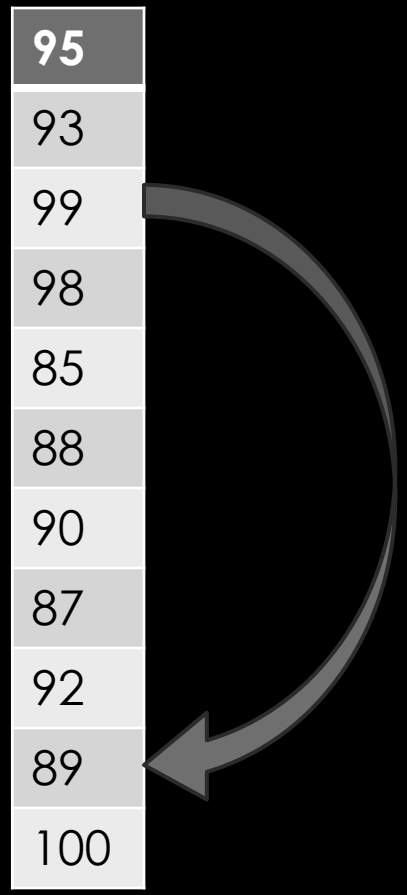
SORTING MARKS



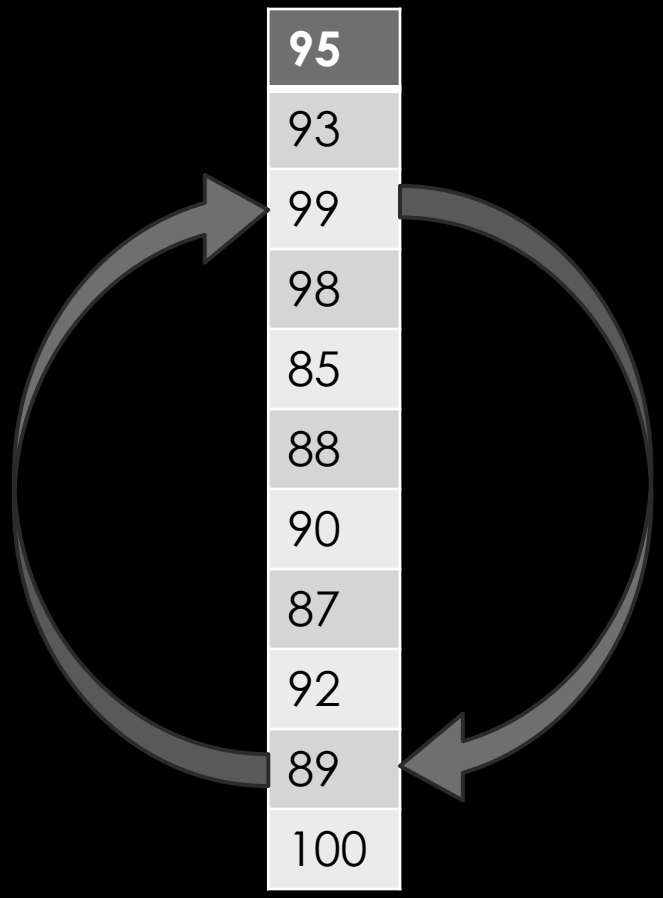
SORTING MARKS



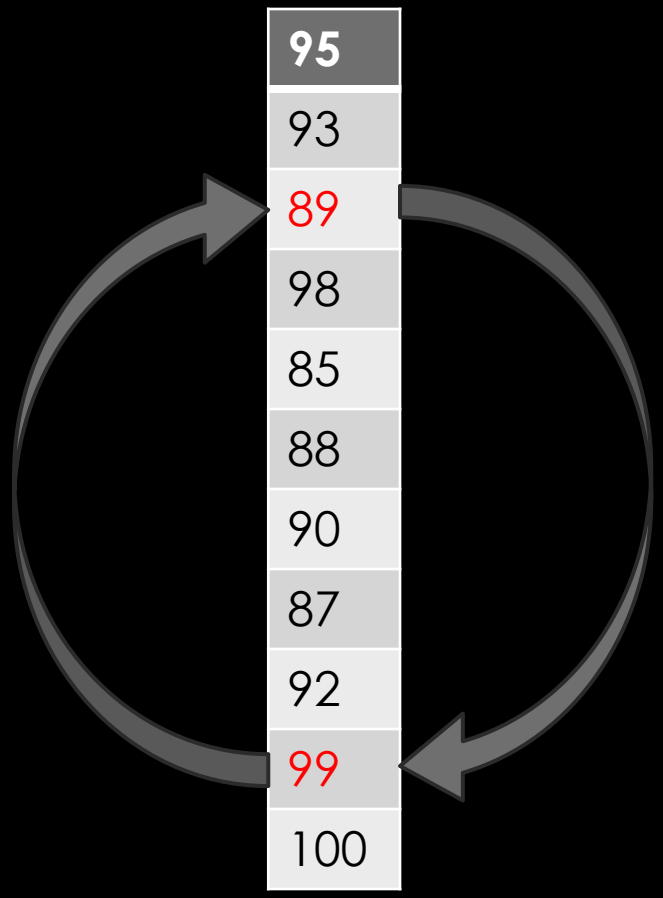
SORTING MARKS



SORTING MARKS



SORTING MARKS



SORTING MARKS

1. Data entry of marks
2. Outer Loop over index `i` for i^{th} max
3. Inner loop to find max in the remaining (index `j 1 to 547-i`)
    1. Swap max with `547-i`
4. Print sorted marks

SORTING MARKS

```
#include <stdio.h>
int main(void){
    double marks[547]={some_list},max; unsigned int max_position;




    for(unsigned int i=0;i<=546;i++)
        printf("%.2lf\n",marks[i]);
    return 0;}
```

SORTING MARKS

```
#include <stdio.h>
int main(void){
    double marks[547]={some_list},max; unsigned int max_position;
    for(unsigned int i=546;i>=1;i--){max=-1;
        for(unsigned int j=0;j<=i;j++){
                                                } //Find max in remaining
        }
                                              //Swap max appropriately
    }
    for(unsigned int i=0;i<=546;i++)
        printf("%.2lf\n",marks[i]);
    return 0;}
```

SORTING MARKS

```
#include <stdio.h>
int main(void){
    double marks[547]={some_list},max; unsigned int max_position;
    for(unsigned int i=546;i>=1;i--){max=-1;
        for(unsigned int j=0;j<=i;j++){
            if(marks[j]>max){max=marks[j];max_position=j;} //Find max in remaining
        }
        marks[i]=max; marks[max_position]=marks[i]; //Swap max appropriately ?
    }
    for(unsigned int i=0;i<=546;i++)
        printf("%.2lf\n",marks[i]);
    return 0;}
```

SORTING MARKS

```
#include <stdio.h>
int main(void){
    double marks[547]={some_list},max; unsigned int max_position;
    for(unsigned int i=546;i>=1;i--){max=-1;
        for(unsigned int j=0;j<=i;j++){
            if(marks[j]>max){max=marks[j];max_position=j;} //Find max in remaining
        }
        marks[max_position]=marks[i];marks[i]=max;  //Swap max appropriately
    }
    for(unsigned int i=0;i<=546;i++)
        printf("%.2lf\n",marks[i]);
    return 0;}
```

MEDIAN OF MARKS

```
// m such that P[X>=m]>=0.5,P[X<=m]>=0.5
```

MEDIAN OF MARKS

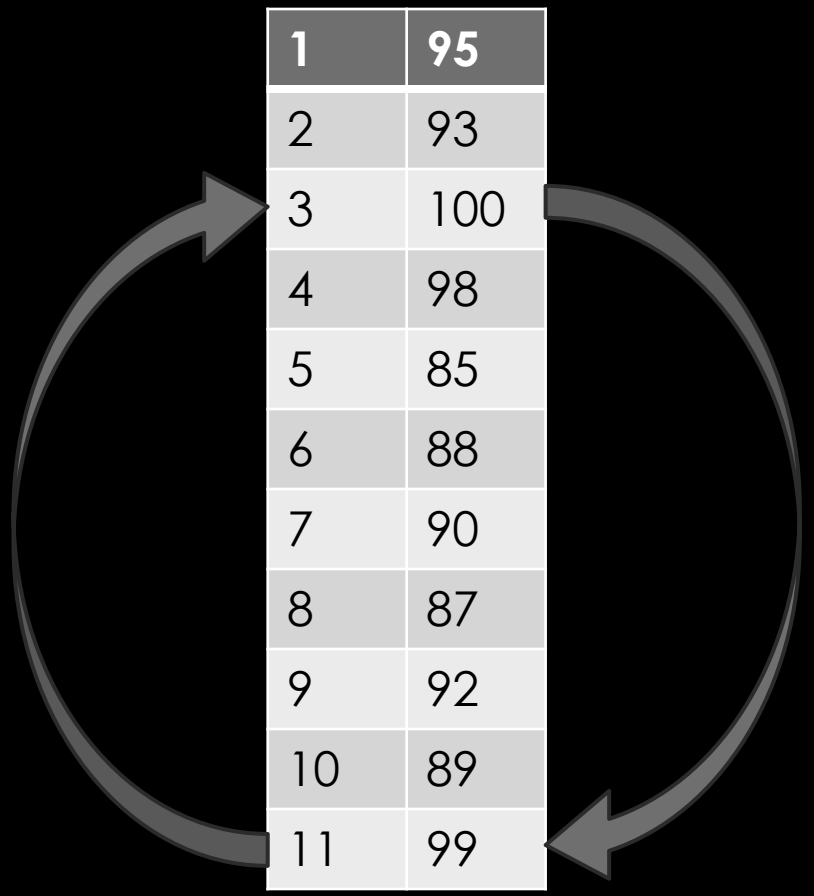
```
#include <stdio.h>
int main(void){
    double marks[547]={some_list},max; unsigned int max_position;
    for(unsigned int i=546;i>=1;i--){max=-1;
        for(unsigned int j=0;j<=i;j++){
            if(marks[j]>max){max=marks[j];max_position=j;}
        }
        marks[max_position]=marks[i];marks[i]=max;
    }
    printf("Median=%.2lf\n",marks[274]);  // m such that P[X>=m]>=0.5,P[X<=m]>=0.5
    return 0;}
```

SORT STUDENTS ACCORDING TO MARKS

SORT STUDENTS ACCORDING TO MARKS

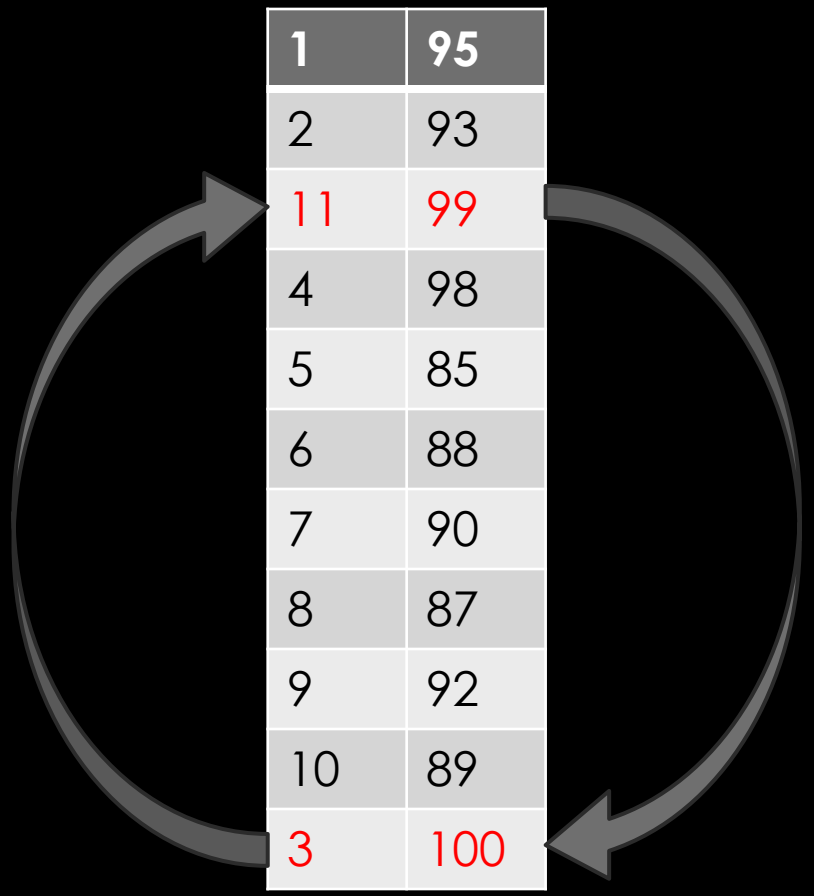
1	95
2	93
3	100
4	98
5	85
6	88
7	90
8	87
9	92
10	89
11	99

SORT STUDENTS ACCORDING TO MARKS



1	95
2	93
3	100
4	98
5	85
6	88
7	90
8	87
9	92
10	89
11	99

SORT STUDENTS ACCORDING TO MARKS



1	95
2	93
11	99
4	98
5	85
6	88
7	90
8	87
9	92
10	89
3	100

SORT STUDENTS ACCORDING TO MARKS

```
#include <stdio.h>
int main(void){
    double marks[547]={marks},max; unsigned int max_position,student_Id[547]={roll},tmp;
    for(unsigned int i=546;i>=1;i--){max=-1;
        for(unsigned int j=0;j<=i;j++){
            if(marks[j]>max){max=marks[j];max_position=j;}
        }
        marks[max_position]=marks[i];marks[i]=max;
        tmp=student_Id[i]; student_Id[i]=student_Id[max_position];student_Id[max_position]=tmp;
    }
    for(unsigned int i=0;i<=546;i++)
        printf("Student %d -> Marks=%.2lf\n",student_Id[i],marks[i]);
    return 0;}
```

HOME WORK

- Read descriptions of few sorting algorithms and try to code them
  - At the very least, Bubble sort (almost same as natural sort algorithm we covered)

SEARCH FOR A KEY IN A DATABASE

SEARCH FOR A KEY IN A DATABASE

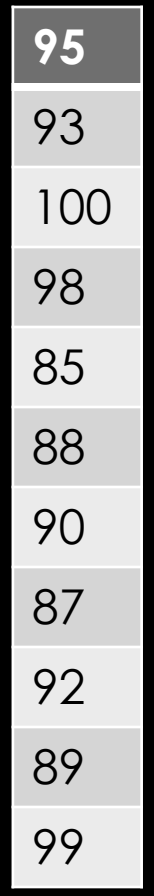
- ASSUME DATA IS INTEGERS AND SO IS THE KEY
- LOOP OVER TO CHECK IF ANY DATUM MATCHES THE KEY

95
93
100
98
85
88
90
87
92
89
99

100

SEARCH FOR A KEY IN A DATABASE

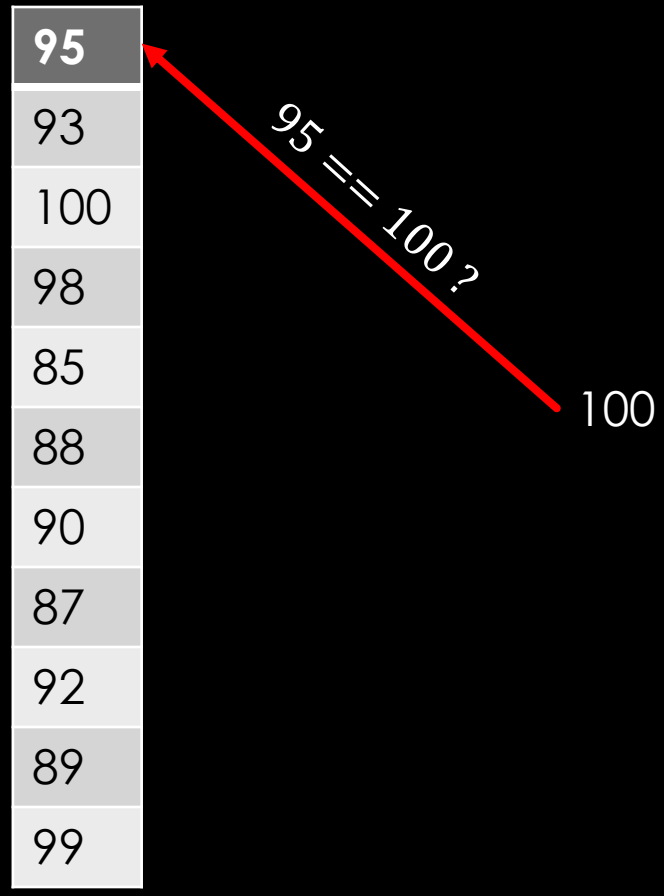
- ASSUME DATA IS INTEGERS AND SO IS THE KEY
- LOOP OVER TO CHECK IF ANY DATUM MATCHES THE KEY



100

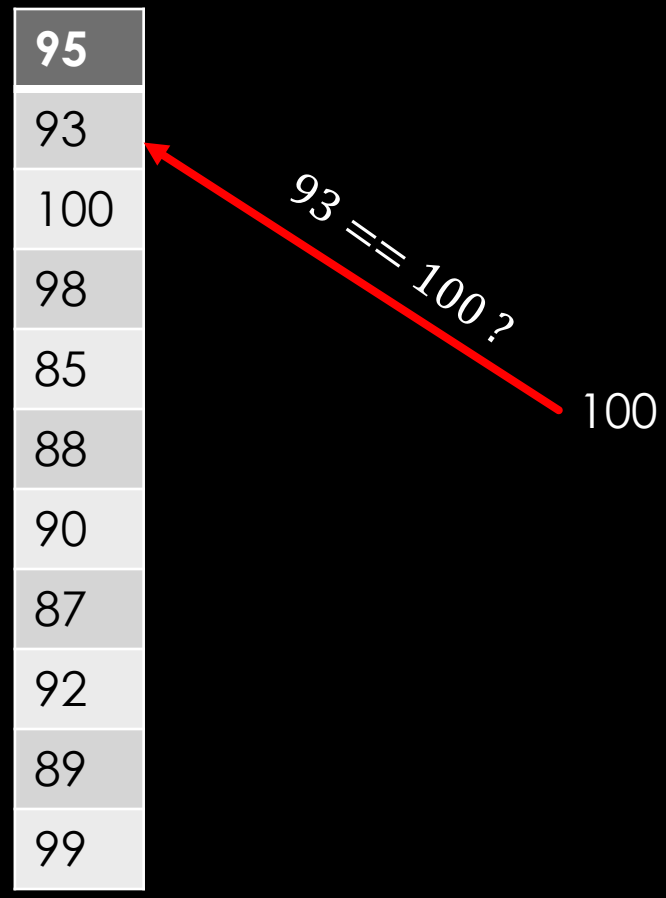
SEARCH FOR A KEY IN A DATABASE

- ASSUME DATA IS INTEGERS AND SO IS THE KEY
- LOOP OVER TO CHECK IF ANY DATUM MATCHES THE KEY



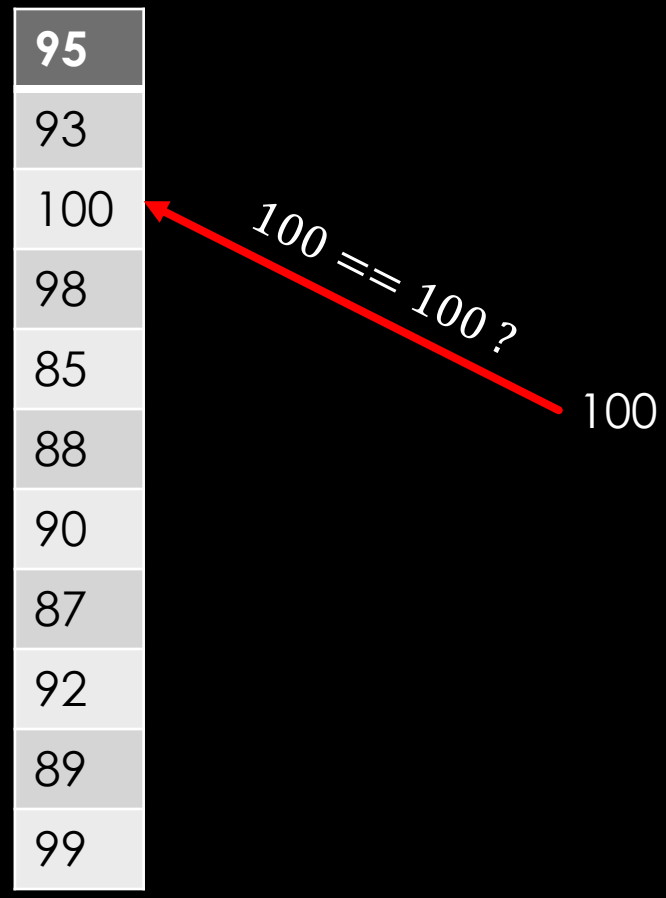
SEARCH FOR A KEY IN A DATABASE

- ASSUME DATA IS INTEGERS AND SO IS THE KEY
- LOOP OVER TO CHECK IF ANY DATUM MATCHES THE KEY



SEARCH FOR A KEY IN A DATABASE

- ASSUME DATA IS INTEGERS AND SO IS THE KEY
- LOOP OVER TO CHECK IF ANY DATUM MATCHES THE KEY



LINEAR SEARCH

```c
#include <stdio.h>
int main(void){
    unsigned int marks[547]={some_list}, key;
    printf("Enter search key: \n"); scanf("%d",&key);









    return 0;}
```

LINEAR SEARCH

```c
#include <stdio.h>
int main(void){
    unsigned int marks[547]={some_list}, key;
    printf("Enter search key: \n"); scanf("%d",&key);
    for(unsigned int i=0;i<=546;i++)
        if(marks[i]==key){
            printf("Key found at index=%d\n",i+1);
            return 0;
        }
    printf("Key absent\n");
    return 0;}
```

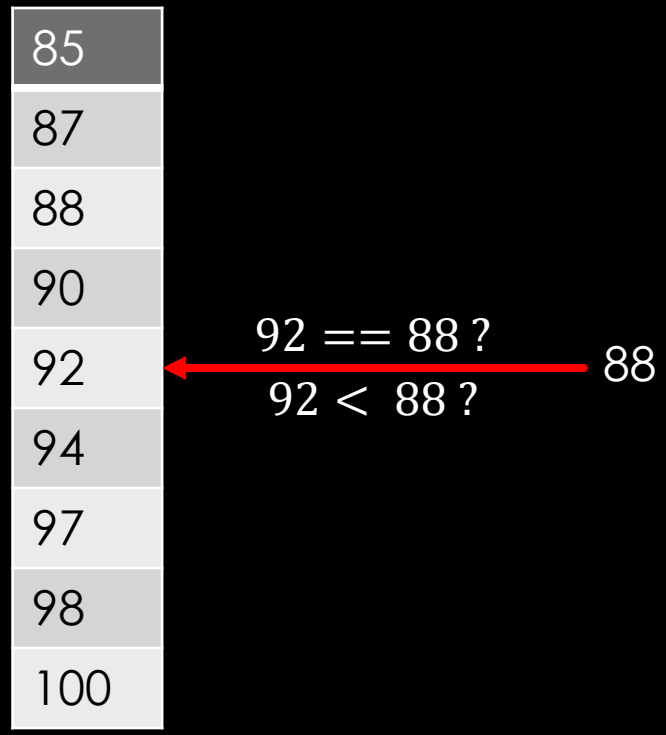
SEARCH ALGORITHM

- Linear search – one comparison eliminated one element
  - Can't be improved, in general
- If numbers are sorted?
  - One inequality may eliminate many!

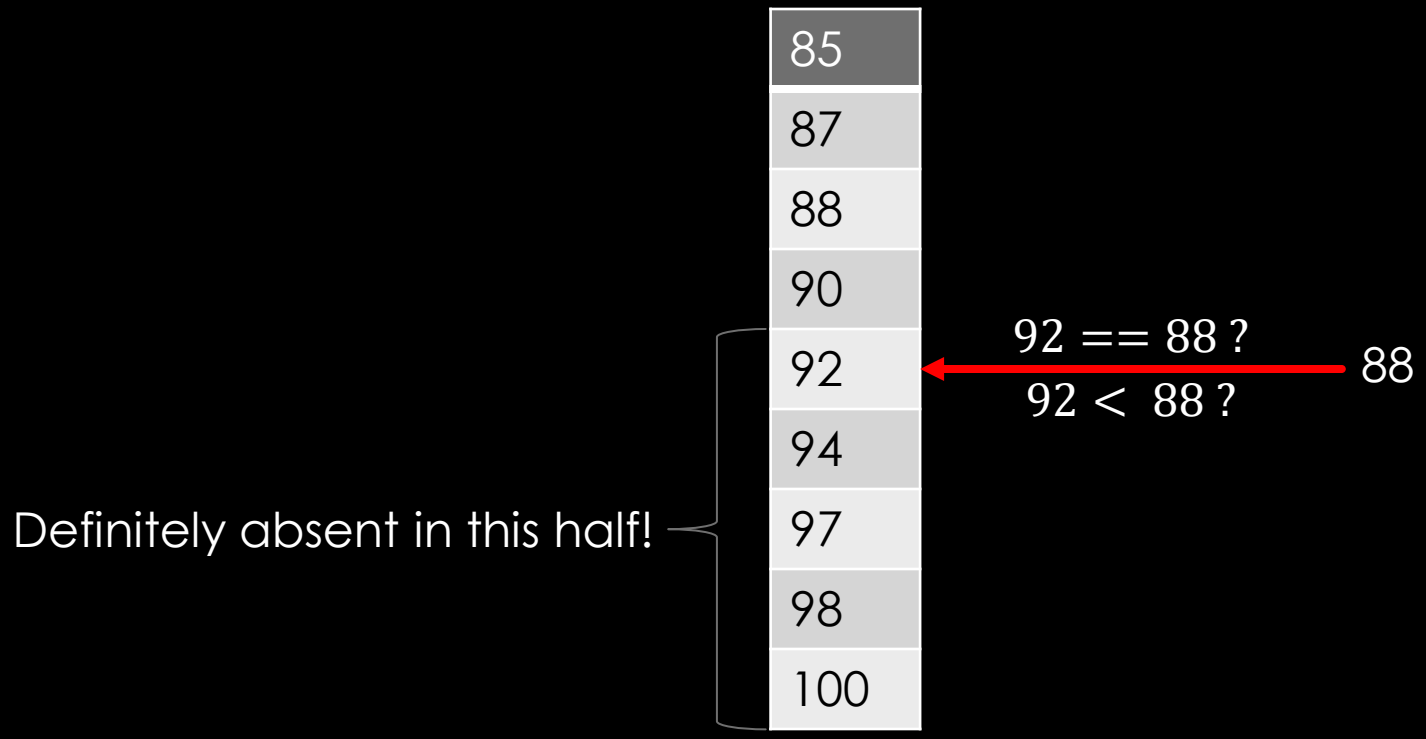
SEARCH ALGORITHM

- Linear search – one comparison eliminated one element
  - Can't be improved, in general
- If numbers are sorted?
  - One inequality may eliminate many!

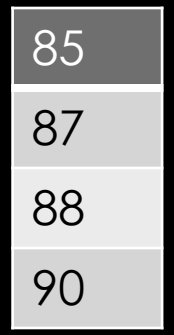
SEARCH FOR A KEY IN A SORTED DATABASE



SEARCH FOR A KEY IN A SORTED DATABASE

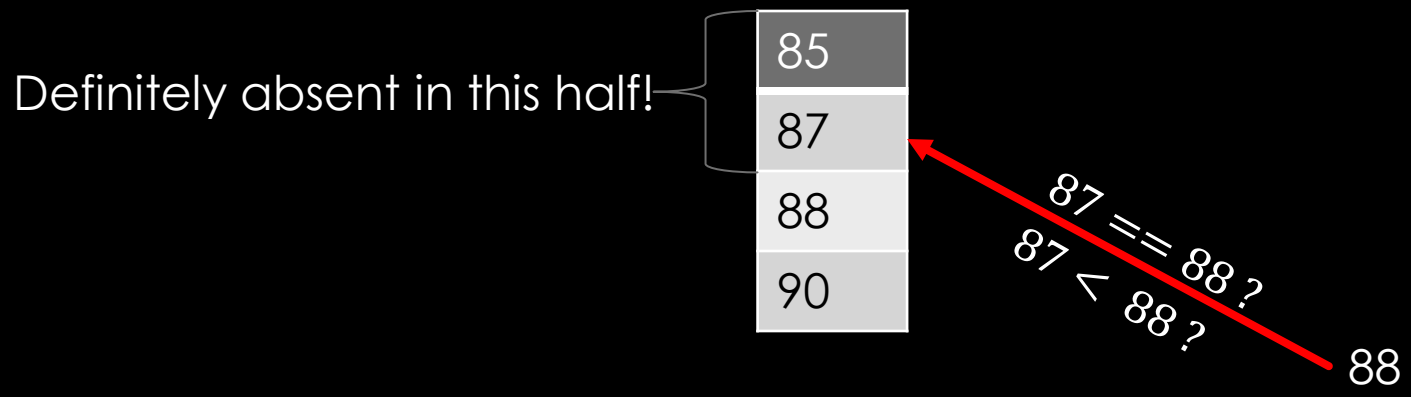


SEARCH FOR A KEY IN A SORTED DATABASE

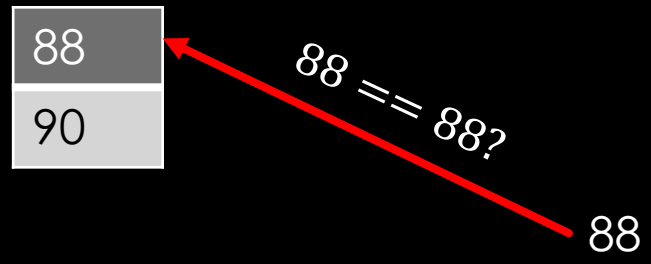


88

SEARCH FOR A KEY IN A SORTED DATABASE



SEARCH FOR A KEY IN A SORTED DATABASE



BINARY SEARCH

- DEFINE SEARCH SPACE BY MAINTAINING TWO INDICES (TOP/LEFT, BOTTOM/RIGHT)
- COMPUTE MIDDLE ELEMENT
- IF MIDDLE==KEY, THEN KEY FOUND, RETURN
- IF MIDDLE<KEY, THEN SHRINK SEARCH SPACE BY LEFT=MIDDLE+1
- IF MIDDLE>KEY, THEN SHRINK SEARCH SPACE BY RIGHT=MIDDLE-1
- REPEAT UNTIL SEARCH SPACE IS EMPTY

BINARY SEARCH

- define search space by maintaining two indices (top/left, bottom/right)
- Compute middle element
- If middle==key, then key found, return
- If middle<Key, then shrink search space by left=middle+1
- If middle>Key, then shrink search space by right=middle-1
- Repeat until search space is empty

BINARY SEARCH

- define search space by maintaining two indices (top/left, bottom/right)
- Compute middle element
- If middle==key, then key found, return
- If middle<Key, then shrink search space by left=middle+1
- If middle>Key, then shrink search space by right=middle-1
- Repeat until search space is empty

BINARY SEARCH

- define search space by maintaining two indices (top/left, bottom/right)
- Compute middle element
- If middle==key, then key found, return
- If middle<Key, then shrink search space by left=middle+1
- If middle>Key, then shrink search space by right=middle-1
- Repeat until search space is empty

BINARY SEARCH

- define search space by maintaining two indices (top/left, bottom/right)
- Compute middle element
- If middle==key, then key found, return
- If middle<Key, then shrink search space by left=middle+1
- If middle>Key, then shrink search space by right=middle-1
- Repeat until search space is empty

BINARY SEARCH

```
#include <stdio.h>
int main(void){
    unsigned int marks[547]={sorted_list}, key,left_id=0,right_id=546,middle;
    printf("Enter search key: \n"); scanf("%d",&key);




    printf("Key absent\n");
    return 0;}
```

BINARY SEARCH

```
#include <stdio.h>
int main(void){
    unsigned int marks[547]={sorted_list}, key,left_id=0,right_id=546,middle;
    printf("Enter search key: \n"); scanf("%d",&key);
    while(left_id<=right_id){           // Repeat until search space is empty
        middle=(left_id+right_id)/2;    // Compute middle index


                                }
    printf("Key absent\n");
    return 0;}
```

BINARY SEARCH

```
#include <stdio.h>
int main(void){
    unsigned int marks[547]={sorted_list}, key,left_id=0,right_id=546,middle;
    printf("Enter search key: \n"); scanf("%d",&key);
    while(left_id<=right_id){
        middle=(left_id+right_id)/2;
        if(marks[middle]==key){printf("Key found at index=%d\n",middle);return 0;}

                                }
    printf("Key absent\n");
    return 0;}
```

BINARY SEARCH

```c
#include <stdio.h>
int main(void){
    unsigned int marks[547]={sorted_list}, key,left_id=0,right_id=546,middle;
    printf("Enter search key: \n"); scanf("%d",&key);
    while(left_id<=right_id){
        middle=(left_id+right_id)/2;
        if(marks[middle]==key){printf("Key found at index=%d\n",middle);return 0;}
        else if(marks[middle]<key) left_id=middle+1;
            else right_id=middle-1;}
    printf("Key absent\n");
    return 0;}
```

char ARRAYS

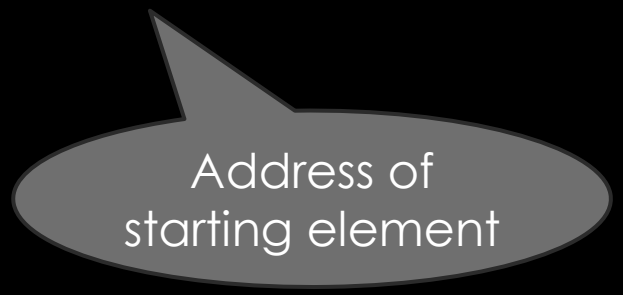
- `char name[]={‘s','a',’c',’h',’i',’n'};`
- `printf("%c",name[3]);`

char ARRAYS - STRINGS!

- `char name[]={'s','a',’c',’h',’i',’n'};`
- `printf("%c",name);`

char ARRAYS - STRINGS!

- `char name[]={'s','a',’c',’h',’i',’n','\0'};`
- `printf("%s",name);`



char ARRAYS - STRINGS!

- `char name[]={'s','a',’c',’h',’i',’n','\0'};`
- `printf("%s",name);`

WHAT'S THE OUTPUT?

```
#include <stdio.h>


int main(void) {
    char name[]={'s','a',’c',’h’,’i',’n','\0’,’t',’e',’n',’d’,
’u’,’l’,’k’,’a’, ’r’,'\0'};
    printf("%s %s",name,name+7);
    return 0;
}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>


int main(void) {
    char name[]={'s','a',’c',’h’,’i',’n','\0’,’t',’e',’n',’d’,
’u’,’l’,’k’,’a’, ’r’,'\0'};
    printf("%s %s",name,name+7); // sachin tendulkar
    return 0;
}
```

char ARRAYS - STRINGS!

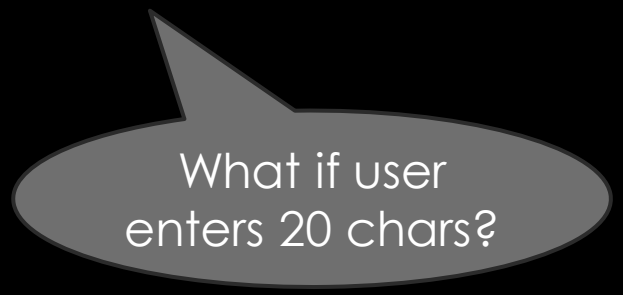
- `char name[15];`
- `scanf("%s",name);`

char ARRAYS - STRINGS!

- `char name[15];`
- `scanf("%s",name);`

char ARRAYS - STRINGS!

- `char name[15];`
- `scanf("%s",name);`



char ARRAYS - STRINGS!

- `char name[15];`
- `scanf("%14s",name);`

Input : abcdefghijklmnop
Output : abcdefghijklmn

MULTI-DIMENSIONAL ARRAYS

TABLES, MATRICES, TENSORS

MULTI-DIMENSIONAL ARRAYS

- `double matrix[15][30];`

MATRIX-VECTOR MULTIPLICATION

```
#include <stdio.h>
int main(void){
    double matrix[2][3]={{1,2,3},{4,5,6}}, vector[3]={7,8,9}, result[2]={0};
    for(unsigned int i=0;i<=1;i++)
        for(unsigned int j=0;j<=2;j++)
            result[i]+=matrix[i][j]*vector[j];
    return 0;
}
```